

蓝桥杯30天算法冲刺集训



Day-25 分块

Note

分块可以解决不少区间处理问题，并且实现起来比较方便，所以还是需要进行学习。本讲义没有附带蓝桥杯的分块真题，因为不少用线段树/树状数组/ST表的题目也可以用分块进行解决。

数列分块就是把数列中每 m 个元素打包起来，达到优化算法的目的。

在前面两节中，我们探讨了树状数组与线段树两种数据结构。树状数组基于二进制划分与倍增思想，线段树基于分治思想。它们之所以能够高效地在一个序列上执行指令并统计信息，就是因为它们把序列中的元素聚合成大大小小的“段”，花费额外的代价对这些“段”进行维护，从而使得每个区间的信息可以快速由几个已有的“段”结合而成。

当然，树状数组与线段树也有其缺点。比如在维护较为复杂的信息（尤其是不满足区间可加、可减性的信息）时显得吃力，代码实现也不是那么简单、直观，需要深入理解并注意许多细节。在本节中，我们将介绍分块算法。分块的基本思想是通过适当的划分，预处理一部分信息并保存下来，用空间换取时间，达到时空平衡。事实上，分块更接近于“朴素”，效率往往比不上树状数组与线段树，但是它更加通用、容易实现。

接下来，我们将介绍分块的方法，可能涉及的几个词语解释如下：

区间：数列中连续一段的元素。

区间操作：将某个区间 $[a, b]$ 的所有元素进行某种改动的操作。

块：我们将数列划分成若干个不相交的区间，每个区间称为一个块。

整块：在一个区间操作时，完整包含于区间的块。

不完整的块：在一个区间操作时，只有部分包含于区间的块，即区间左右端点所在的两个块。

简介

其实，分块是一种思想，而不是一种数据结构。

分块的基本思想是，通过对原数据的适当划分，并在划分后的每一个块上预处理部分信息，从而较一般的暴力算法取得更优的时间复杂度。

分块的时间复杂度主要取决于分块的块长，一般可以通过均值不等式求出**某个问题下的最优块长**，以及相应的时间复杂度。

分块是一种很灵活的思想，相较于树状数组和线段树，分块的优点是通用性更好，可以维护很多树状数组和线段树无法维护的信息。

当然，分块的缺点是渐进意义的复杂度，相较于线段树和树状数组不够好。

不过在大多数问题上，分块仍然是解决这些问题的一个不错选择。

下面是几个例子。

区间修改，单点查询

思路

这是一道能用许多数据结构优化的经典题，可以用于不同数据结构训练。

数列分块就是把数列中每 m 个元素打包起来，达到优化算法的目的。

以此题为例，如果我们把每 m 个元素分为一块，共有 $\lceil n/m \rceil$ 块，每次区间加的操作会涉及 $O(n/m)$ 个整块，以及区间两侧两个不完整的块中至多 $2m$ 个元素。

我们给每个块设置一个加法标记（就是记录这个块中元素一起加了多少），每次操作对每个整块直接 $O(1)$ 标记，而不完整的块由于元素比较少，暴力修改元素的值。

每次询问时返回元素的值加上其所在块的加法标记。

这样每次操作的复杂度是 $O(n/m) + O(m)$ ，根据均值不等式，当 m 取 $\sqrt{n}$ 时总复杂度最低，为了方便，我们都默认下文的分块大小为 $\sqrt{n}$ 。

```

#include <bits/stdc++.h>
using namespace std;

const int N = 1e5+5;
int n, op, l, r, c, a[N];
int sum[1000], add[1000], len;
int main(){
    cin >> n;
    for(int i = 1; i <= n; i++)
        cin >> a[i];
    int s = sqrt(n); // 块的最优长度
    for(int i = 1; i <= n; i++)
    {
        cin >> op >> l >> r >> c;
        if(op) cout << a[r] + add[(r-1)/s+1] << '\n';
        else {
            // 计算左右端点所在的块的编号
            int sl = (l-1)/s+1, sr = (r-1)/s+1;
            if(sl == sr) { // l, r 在一个块内
                for(int i = l; i <= r; i++)
                    a[i] += c;
                continue;
            }
            // 左边不完整的块单独处理
            for(int i = l; i <= sl*s; i++)
                a[i] += c;
            // 完整的块，整体增加
            for(int i = sl*s+1; i < sr*s; i++)
                add[i] += c;
            // 右边不完整的块单独处理
            for(int i = (sr-1)*s+1; i <= r; i++)
                a[i] += c;
        }
    }
    return 0;
}

```

另一种写法

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5+5, M = 1e3+5;
int n, op, ql, qr, c, a[N];
// id[i]: a[i] 对应块编号
// add: 块信息      l,r: 块左右端点      siz: 块大小
int s, num, id[N], add[M], l[M], r[M], siz[M];

inline void build() {
    s = sqrt(n); // 块的最优长度
    num = ceil(1.0*n/s);
    for(int i = 1; i <= num; i++) { // 当 s=sqrt(n) 时, n/s =
s
        l[i] = (i-1) * s + 1;
        r[i] = i*s;
    }
    r[num] = n; // 最后一个区间可能不完整
    for(int i = 1; i <= num; i++) {
        siz[i] = r[i] - l[i] + 1; // 区间长度, 在维护有些块信息
时会用到
        for(int j = l[i]; j <= r[i]; j++) {
            id[j] = i;
            add[i] = 0;
        }
    }
}

int query(int x) {
    return a[x] + add[id[x]];
}

void modify(int ql, int qr, int c) {
    int sid = id[ql], eid = id[qr];
    if(sid == eid) { // l, r 在一个块内
        for(int i = ql; i <= qr; i++)
```

```

        a[i] += c;
    return;
}
for(int i = ql; i <= r[sid]; i++) // 左边不完整的块单独处理
    a[i] += c;
for(int i = sid+1; i < eid; i++) // 完整的块，整体增加
    add[i] += c;
for(int i = l[eid]; i <= qr; i++) // 右边不完整的块单独处理
    a[i] += c;
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n;
    for(int i = 1; i <= n; i++)
        cin >> a[i];
    build();

    for(int i = 1; i <= n; i++)
    {
        cin >> op >> ql >> qr >> c;
        if(op) cout << query(qr) << '\n';
        else {
            modify(ql, qr, c);
        }
    }
    return 0;
}

```

区间修改，区间查询

思路

这题的询问变成了区间上的询问，不完整的块还是暴力；而要想快速统计完整块的答案，需要维护每个块的元素和，先要预处理一下。

考虑区间修改操作，不完整的块直接改，同时也别忘了更新块的元素和；完整的块类似之前标记的做法，直接根据块的元素和所加的值计算元素和的增量。

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e6 + 5, M = 1e4 + 5;
long long n, op, ql, qr, c, a[N];
// id[i]: a[i] 对应块编号
// tag: 块信息      l,r: 块左右端点      siz: 块大小
long long s, num, id[N], tag[M], l[M], r[M], siz[M], sum[M];

inline void build() {
    s = sqrt(n); // 块的最优长度
    num = n / s; // 块的数量

    for (int i = 1; i <= num; i++) { // 当 s=sqrt(n) 时, n/s = s
        l[i] = (i - 1) * s + 1;
        r[i] = i * s;
    }

    r[num] = n; // 最后一个区间可能不完整

    for (int i = 1; i <= num; i++) {
        siz[i] = r[i] - l[i] + 1; // 区间长度, 在维护有些块信息时会用到

        for (int j = l[i]; j <= r[i]; j++) {
            id[j] = i;
            sum[i] += a[j];
            tag[i] = 0;
        }
    }
}

int query(int ql, int qr, long long c) {
    int sid = id[ql], eid = id[qr];
```

```

    long long res = 0;
    if (sid == eid) {
        for (int i = ql; i <= qr; i++)
            res += a[i] + tag[sid];
    } else {
        for (int i = ql; i <= r[sid]; i++) {
            res += a[i] + tag[sid];
        }
        for (int i = sid + 1; i < eid; i++) {
            res += sum[i];
        }
        for (int i = l[eid]; i <= qr; i++) {
            res += a[i] + tag[eid];
        }
    }
    return res % (c+1);
}

void modify(int ql, int qr, int c) {
    int sid = id[ql], eid = id[qr];
    if (sid == eid) { // l, r 在一个块内
        for (int i = ql; i <= qr; i++)
            a[i] += c, sum[id[ql]] += c;
    } else {
        for (int i = ql; i <= r[sid]; i++) // 左边不完整的块单独
            a[i] += c, sum[sid] += c;

        for (int i = sid + 1; i < eid; i++) // 完整的块，整体增
            {
                tag[i] += c;
                sum[i] += c * siz[i];
            }

        for (int i = l[eid]; i <= qr; i++) // 右边不完整的块单独
            a[i] += c, sum[eid] += c;
    }
}

```



```

}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n;

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    build();

    for (int i = 1; i <= n; i++) {
        cin >> op >> ql >> qr >> c;
        if (op)
            cout << query(ql, qr, c) << '\n';
        else {
            modify(ql, qr, c);
        }
    }

    return 0;
}

```

习题

数列分块入门1

数列分块入门2